Department of Computer Science and Engineering
University of Moratuwa

CS4533 – Concurrent Programming

Lab 03: OpenMP — Practical Parallel Programming

Module:	CS4533 – Concurrent Programming
Lab:	Lab 03 – OpenMP: Practical Parallel Programming
Duration:	2 hours (in-class)
Topics:	OpenMP directives, parallel for, reduction, scheduling, tasks

Prerequisites

- Completed Labs 01 and 02 (threads, synchronization)
- GCC with OpenMP support (included by default)
- Compile all exercises with: gcc -fopenmp -O0 -o output source.c
- The -fopenmp flag enables OpenMP; without it, #pragma omp directives are silently ignored

Learning Objectives

By the end of this lab, you should be able to:

- Write parallel programs using OpenMP compiler directives
- Understand shared vs private variables in OpenMP
- Use parallel for, critical, atomic, and reduction for data-parallel computation
- Compare static, dynamic, and guided scheduling policies
- Use OpenMP tasks for irregular parallelism (divide-and-conquer)

OpenMP Quick Reference

OpenMP uses compiler directives (#pragma omp) instead of manual thread management. Here are the key constructs you will need in this lab:

```
// Create a parallel region (spawns threads)
#pragma omp parallel { ... }

// Parallelize a for loop
#pragma omp parallel for

// Data sharing: shared(var) vs private(var)
#pragma omp parallel for private(x) shared(arr)

// Protect a critical section (one thread at a time)
#pragma omp critical
```

```
// Atomic update (hardware-level, faster than critical)
#pragma omp atomic

// Reduction (private accumulation, combined at the end)
#pragma omp parallel for reduction(+:sum)

// Scheduling policies
#pragma omp parallel for schedule(static)
#pragma omp parallel for schedule(dynamic)
#pragma omp parallel for schedule(guided)

// Task parallelism (for recursive/irregular work)
#pragma omp task
#pragma omp taskwait
#pragma omp single  // only one thread executes this block

// Useful functions
omp_get_thread_num()     // current thread ID
omp_get_num_threads()    // total threads in region
omp_set_num_threads(n)   // set thread count
omp_get_wtime()          // wall clock time (for timing)
```

Note: rand() is NOT thread-safe. Use rand_r(&seed) with a per-thread seed inside parallel regions.

Exercise 1: OpenMP Warm-up

Background

Unlike PThreads, where you manually create and manage threads, OpenMP uses compiler directives to automatically parallelize code. The runtime handles thread creation, distribution of work, and synchronization. This exercise gets you comfortable with the basic syntax.

Task A: Hello World

Write a program that creates a parallel region where each thread prints its ID and the total thread count. Run with 1, 2, 4, and 8 threads using omp_set_num_threads() or the OMP_NUM_THREADS environment variable.

Task B: Shared vs Private

Write a program that demonstrates the difference between shared and private variables. Declare two variables before a parallel region. Make one shared and one private. Inside the region, each thread should print both variables and then modify the private one. After the parallel region, print both variables again in main. Observe what value the private variable has inside each thread at the start of the region.

Task C: Parallel For

Write a program that fills an array of 20 elements with the square of each index (arr[i] = i * i). Parallelize the loop with #pragma omp parallel for. Inside the loop, print which thread computed which index. After the loop, verify the results are correct.

Questions

Q1.1: In Task B, what value does a private variable have inside each thread at the start of the parallel region? Why? What value does it have in main after the region ends?

Q1.2: In Task C, how does OpenMP divide the 20 iterations among 4 threads? Which thread computes which indices? Is this division always the same across runs?

Exercise 2: Monte Carlo Pi Estimation

Background

Monte Carlo methods use randomness to estimate numerical values. To estimate pi: imagine a unit square [0,1] x [0,1] with a quarter circle of radius 1 inscribed in it. Randomly throw darts at the square. The ratio of darts landing inside the quarter circle to total darts approximates pi/4. A point (x, y) is inside the quarter circle if x*x + y*y <= 1.0.

Serial Code

Here is a working serial implementation. Compile, run, and verify it works before attempting to parallelize.

```
#include <stdio.h>
#include <stdlib.h>
#include <omp.h>

#define NUM_DARTS 10000000

int main() {
    int count = 0;
    srand(42);

    double start = omp_get_wtime();

    for (int i = 0; i < NUM_DARTS; i++) {
        double x = (double)rand() / RAND_MAX;
        double y = (double)rand() / RAND_MAX;
        if (x * x + y * y <= 1.0)
            count++;
    }

    double end = omp_get_wtime();
    double pi = 4.0 * count / NUM_DARTS;

    printf("Pi = %.6f (error: %.6f)\n", pi, pi - 3.141593);
    printf("Time: %.4f seconds\n", end - start);
    return 0;
}
```

Your Task

Parallelize the above code using OpenMP. You will encounter a race condition on the shared counter. Fix it using four different approaches, each implemented as a separate function:

- Approach A: #pragma omp critical
- Approach B: #pragma omp atomic
- Approach C: The reduction clause
- Approach D: Manual per-thread local counters (using thread ID to index into an array), combined after the parallel region

Compare all four approaches in terms of correctness and execution time with 4 threads. Then run the best approach with 1, 2, 4, and 8 threads and measure speedup.

Hints

- rand() is not thread-safe. Use rand_r(&seed) with a per-thread seed, e.g., unsigned int seed = 42 + omp_get_thread_num();
- The seed must be private to each thread — think about where to declare it.

Questions

Q2.1: Present a table comparing all four approaches (4 threads): pi estimate, error, and execution time. Which is fastest? Which is slowest? Explain why.

Q2.2: Why is reduction faster than critical? How does it relate to the local accumulation pattern from Lab 1, Exercise 3?

Q2.3: Increase NUM_DARTS to 100,000,000. Does pi accuracy improve? Does the speedup from parallelism change? Explain.

Exercise 3: Image Processing — Box Blur Filter

Background

A box blur is a simple image processing operation where each output pixel is the average of its surrounding neighbours in a square kernel. For a 3x3 blur, each pixel is replaced by the average of the 9 pixels in its 3x3 neighbourhood. This is naturally parallelizable because each output pixel can be computed independently.

Serial Code

Here is a working serial implementation of the box blur:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <omp.h>

#define WIDTH  2000
#define HEIGHT 2000
#define KERNEL 3

int input[HEIGHT][WIDTH];
int output_serial[HEIGHT][WIDTH];
int output_parallel[HEIGHT][WIDTH];

void generate_image() {
    srand(42);
```

```
    for (int i = 0; i < HEIGHT; i++)
        for (int j = 0; j < WIDTH; j++)
            input[i][j] = rand() % 256;
}

void blur_serial() {
    int half = KERNEL / 2;
    for (int i = half; i < HEIGHT - half; i++) {
        for (int j = half; j < WIDTH - half; j++) {
            int sum = 0;
            for (int di = -half; di <= half; di++)
                for (int dj = -half; dj <= half; dj++)
                    sum += input[i + di][j + dj];
            output_serial[i][j] = sum / (KERNEL * KERNEL);
        }
    }
}

int verify() {
    for (int i = 0; i < HEIGHT; i++)
        for (int j = 0; j < WIDTH; j++)
            if (output_serial[i][j] != output_parallel[i][j])
                return 0;
    return 1;
}

int main() {
    generate_image();
    double start, end;

    start = omp_get_wtime();
    blur_serial();
    end = omp_get_wtime();
    printf("Serial: %.4f seconds\n", end - start);

    // TODO: Call your parallel blur function here
    // TODO: Time it with different thread counts
    // TODO: Verify output matches serial version

    return 0;
}
```

Your Task

- Write a blur_parallel(int num_threads) function that parallelizes the blur using OpenMP.
- Write the output to output_parallel[][] and use the verify() function to confirm your parallel version produces identical results to the serial version.
- Measure speedup with 1, 2, 4, and 8 threads.
- Experiment with different scheduling policies: static, dynamic, dynamic(64), and guided. Run each with 4 threads and compare execution times.

Questions

Q3.1: Present a table showing execution time, speedup, and parallel efficiency (efficiency = speedup / num_threads) for 1, 2, 4, and 8 threads with static scheduling.

Q3.2: Present a table comparing the four scheduling policies with 4 threads. Which performs best for this workload? Explain why.

Q3.3: For this box blur, each row takes roughly the same time to process (uniform workload). Under what kind of workload would dynamic scheduling significantly outperform static? Describe a concrete example.

Exercise 4: Parallel Merge Sort with Tasks

Background

Not all parallelism fits the parallel-for pattern. Divide-and-conquer algorithms like merge sort create work dynamically through recursion. OpenMP tasks let you express this: #pragma omp task packages a block of code for potential execution by another thread. The #pragma omp taskwait directive waits for all child tasks to finish before continuing.

Serial Code

Here is a working serial merge sort:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <omp.h>

#define SIZE 10000000

void merge(int *arr, int *temp, int left, int mid,
           int right) {
    int i = left, j = mid + 1, k = left;
    while (i <= mid && j <= right) {
        if (arr[i] <= arr[j])
            temp[k++] = arr[i++];
        else
            temp[k++] = arr[j++];
    }
    while (i <= mid)  temp[k++] = arr[i++];
    while (j <= right) temp[k++] = arr[j++];
    memcpy(arr + left, temp + left,
           (right - left + 1) * sizeof(int));
}

void merge_sort(int *arr, int *temp,
                int left, int right) {
    if (left >= right) return;
    int mid = (left + right) / 2;
    merge_sort(arr, temp, left, mid);
    merge_sort(arr, temp, mid + 1, right);
    merge(arr, temp, left, mid, right);
}
```

```
int is_sorted(int *arr, int n) {
    for (int i = 1; i < n; i++)
        if (arr[i] < arr[i - 1]) return 0;
    return 1;
}

int main() {
    int *arr = malloc(SIZE * sizeof(int));
    int *temp = malloc(SIZE * sizeof(int));
    int *backup = malloc(SIZE * sizeof(int));

    srand(42);
    for (int i = 0; i < SIZE; i++)
        arr[i] = rand();
    memcpy(backup, arr, SIZE * sizeof(int));

    double start, end;

    // Serial sort
    start = omp_get_wtime();
    merge_sort(arr, temp, 0, SIZE - 1);
    end = omp_get_wtime();
    printf("Serial: %.4f seconds, sorted: %s\n",
           end - start,
           is_sorted(arr, SIZE) ? "YES" : "NO");

    // TODO: Reset array from backup
    // TODO: Implement and call parallel merge sort
    // TODO: Time it with different thread counts
    // TODO: Verify it is correctly sorted

    free(arr); free(temp); free(backup);
    return 0;
}
```

Your Task

- Write a merge_sort_parallel() function that uses #pragma omp task for the two recursive calls and #pragma omp taskwait before the merge step.
- Add a depth threshold parameter: beyond a certain recursion depth (e.g., depth > 10), fall back to the serial merge_sort to avoid creating too many tiny tasks.
- Think carefully about how to launch the parallel sort. Only one thread should make the initial call — look into #pragma omp parallel and #pragma omp single.
- Verify the result is correctly sorted using is_sorted().
- Measure serial vs parallel time with 1, 2, 4, and 8 threads.

Questions

Q4.1: Present a table showing serial time, parallel time, and speedup with 1, 2, 4, and 8 threads.

Q4.2: What happens if you remove the depth threshold and create tasks all the way down to single elements? Test with a smaller array (SIZE = 100000) and report the result. Why is task granularity important?

Q4.3: Explain why #pragma omp single is needed before the first call to the parallel sort function. What goes wrong if you remove it? Test and describe the behaviour.

Submission Guidelines

- Submit source files (.c), one per exercise, named as: Lab03_<IndexNumber1>_<IndexNumber2>_Ex<N>.c
- For Exercise 1, submit a single file containing all three tasks.
- Submit one PDF report containing: terminal output, performance tables, speedup calculations, and answers to all questions.
- Compile all exercises with: gcc -fopenmp -O0 -o output <Name>.c
- Submit via Moodle before the deadline.
